

Project02 Wiki

Information

강의 명 : 운영체제 (12299)

과제 명 : ELE3021_Project #2

작성자 : 컴퓨터소프트웨어학부 2020003309 최성민

Design

Process Management

- command line

sh.c를 참조하여 구현합니다.

- list

현재 돌아가는 process 들을 출력하는 system call을 구현합니다.

state가 RUNNING, RUNNABLE, SLEEPING인 process만을 출력합니다.

thread는 출력하지 않습니다.

- kill

kill 함수를 이용하여 구현합니다.

- execute <path> <stacksize>

exec2 함수를 이용하여 구현합니다.

실행되는 프로세스의 출력 스트림은 그대로 유지합니다.

1. 따로 output을 파일로 저장하는 기능의 구현이 명세에 존재하지 않습니다.
2. process manager를 사용하는데에 크게 문제가 없음을 체감하였습니다.

3. 프로세스의 출력이 process manager 실행 중에 나오는 것이 부자연스러울 수 있지만,
출력 스트림을 막는 것 보다 더 유용하다고 판단했습니다.

- memlim

setmemorylimit 함수를 이용하여 구현합니다.

Light-weight Process (LWP)

- thread

운영체제는 thread를 process와 비슷하게 취급합니다.

thread를 생성한 process는 main thread로 취급받습니다.

thread는 main thread와 pgdir를 공유합니다.

thread는 자신만의 1 page size의 stack memory를 가지고 있습니다. (가드페이지 별도)

thread는 stack memory 가장 위에 fake return PC를 가지고 있습니다.

시작 함수의 끝에 도달하여 종료하는 경우는 고려하지 않습니다.

thread의 pid는 main thread와 같습니다.

thread는 tid라는 자신을 구분할 수 있는 값을 가지고 있습니다.

thread는 main thread를 tmain에 저장하여 지니고 있습니다.

thread는 retval라는 변수에 start_routine의 return값을 저장합니다.

thread는 spgn이라는 변수에 stack page 개수를 저장합니다. 생성 시 1이겠죠? (가드페이지 별도)

- thread create

thread는 해당 함수를 통해 생성됩니다.

fork 함수와 exec 함수를 참조합니다.

thread는 main thread로부터 pgdir를 공유합니다.

thread는 stack memory 를 새로 할당받고, fake return pc와 arg를 저장합니다.

thread는 eip를 start_routine으로 지정합니다.

- thread exit

thread는 해당 함수를 통해 정상 종료됩니다.

exit 함수를 참조합니다.

thread는 부모가 아닌 main thread에게 자원 회수를 요청합니다.

- thread join

thread는 해당 함수를 통해 정상적으로 자원이 회수됩니다.

wait 함수를 참조합니다.

main thread는 해당 함수를 통해 thread의 자원을 회수합니다.

thread와 pgdir를 공유 중이므로, pgdir는 free시키지 않습니다.

- thread clear

thread는 해당 함수를 통해 관계된 thread를 정리합니다.

caller process(thread)를 제외한 관계있는 thread는 모두 타겟 thread입니다.

caller는 타겟 thread의 상태를 CLEANING으로 바꾸어 스케줄링되지 않게 합니다.

caller는 타겟 thread를 대상으로 기본적인 exit 과정과 자원 회수를 진행합니다.

caller process(thread)는 process(main thread)가 됩니다.

- thread sync

thread는 해당 함수를 통해 관계된 thread와 동기화합니다.

관계있는 모든 thread의 값을 타겟 프로세스의 값으로 동기화시킵니다.

Implement

Process Management

- command line

```
118 int
119 main(int argc, char *argv[])
120 {
121     static char buf[MAX_BUF];
122     char args[MAX_ARGS][MAX_ARG_LEN];
123     char cmd[MAX_ARG_LEN];
124     int arglen = -1, flag = 0;
125
126     for(;;) {
127         if (getbuf(buf, sizeof(buf)) < 0) continue;
128
129         if ((arglen = splitstr(buf, cmd, args)) < 0) {
130             printf(1, "the input value is not in a valid format\n\n");
131             continue;
132         }
133
134         if ((flag = execcmd(cmd, args, arglen)) < 0) {
135             printf(1, "the input value is not in a valid format\n\n");
136         } else if (flag) {
137             break;
138         }
139     }
140     printf(1, "exit\n");
141     exit();
142 }
```

main 함수입니다. getbuf 함수와 splitstr 함수를 통해 적절한 형식으로 변환합니다.

```
25 int
26 getbuf(char *buf, int nbuf)
27 {
28     printf(2, "> ");
29     memset(buf, 0, nbuf);
30     gets(buf, nbuf);
31     if(buf[0] == 0) // EOF
32         return -1;
33     return 0;
34 }
```

getbuf 함수입니다. gets 함수를 통해 buffer에 입력 스트림을 받아줍니다.

```
37 int
38 splitstr(char *buf, char *cmd, char args[MAX_ARGS][MAX_ARG_LEN])
39 {
40     memset(cmd, 0, MAX_ARG_LEN);
41     memset(args, 0, MAX_ARGS * MAX_ARG_LEN * sizeof(char));
42
43     int i, splitlen = -1, start = 0, maxlen = strlen(buf);
44
45     for(i = 0; i < maxlen; i++) {
46         if (buf[i] == ' ' || buf[i] == '\n') {
47             if (splitlen >= MAX_ARGS) return -1;
48             if (i - start >= MAX_ARG_LEN) return -1;
49
50             if (splitlen < 0) {
51                 strncpy(cmd, buf + start, i - start);
52             } else {
53                 strncpy(args[splitlen], buf + start, i - start);
54             }
55             splitlen++;
56             start = i + 1;
57         }
58     }
59
60     return splitlen;
61 }
```

splitstr 함수입니다. buffer의 내용을 적절하게 command와 args로 나눠줍니다.

- list

execcmd의 list 부분입니다.

```

64 int
65 execcmd(char *cmd, char args[MAX_ARGS][MAX_ARG_LEN], int arglen)
66 {
67     int ppid = getpid();
68
69     if (!strcmp(cmd, "list")) {
70         proclst();
71     }

```

proclist라는 system call을 호출해줍니다.

```

657 void
658 proclist(void)
659 {
660     struct proc *p;
661
662     char namestr[DSTLEN + 1] = {0};
663     char pidstr[DSTLEN + 1] = {0};
664     char spstr[DSTLEN + 1] = {0};
665     char szstr[DSTLEN + 1] = {0};
666     char szlimstr[DSTLEN + 1] = {0};
667
668     padset(namestr, "name", ' ');
669     padset(pidstr, "pid", ' ');
670     padset(spstr, "stack pagenum", ' ');
671     padset(szstr, "mem size", ' ');
672     padset(szlimstr, "mem size limit", ' ');
673     printf("| %s | %s | %s | %s | %s |\n", namestr, pidstr, spstr, szstr, szlimstr);
674
675     padset(namestr, "", '-');
676     padset(pidstr, "", '-');
677     padset(spstr, "", '-');
678     padset(szstr, "", '-');
679     padset(szlimstr, "", '-');
680     printf("| %s | %s | %s | %s | %s |\n", namestr, pidstr, spstr, szstr, szlimstr);
681
682     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
683         if(p->state != RUNNING &&
684            p->state != RUNNABLE &&
685            p->state != SLEEPING)
686             continue;
687         if(p->tid) continue;
688
689         padset(namestr, p->name, ' ');
690         padset(pidstr, intToStr(p->pid, pidstr), ' ');
691         padset(spstr, intToStr(p->spgn, spstr), ' ');
692         padset(szstr, intToStr(p->sz, szstr), ' ');
693         padset(szlimstr, intToStr(p->szlim, szlimstr), ' ');
694         printf("| %s | %s | %s | %s | %s |\n", namestr, pidstr, spstr, szstr, szlimstr);
695     }
696     printf("\n");
697 }

```

process의 state와 thread 여부에 따라 출력을 진행합니다.

- kill

```

71
72 else if (!strcmp(cmd, "kill")) {
73     if (arglen < 1) return -1;
74
75     int pid = atoi(args[0]);
76
77     if (pid == getpid()) {
78         printf(1, "(pid:%d) process killed\n\n", pid);
79         return 1;
80     } else if (kill(pid) < 0) {
81         printf(1, "(pid:%d) process not found\n\n", pid);
82     } else {
83         printf(1, "(pid:%d) process killed\n\n", pid);
84     }
85 }

```

kill 함수를 통해 pid process를 강제 종료시킵니다.

이때, pid가 자기 자신일 경우, 자연스러운 종료를 위해 exit 로직을 따라갑니다.

```

133
134 if ((flag = execcmd(cmd, args, arglen)) < 0) {
135     printf(1, "the input value is not in a valid format\n\n");
136 } else if (flag) {
137     break;
138 }
139
140 printf(1, "exit\n");
141 exit();
142

```

main 함수 부분입니다.

execcmd의 return 값이 1일 경우 exit를 진행합니다.

- execute <path> <stacksize>

```

86 else if (!strcmp(cmd, "execute")) {
87     if (arglen < 2) return -1;
88
89     int stacksize = atoi(args[1]);
90     char* path = args[0];
91     char* argv[] = {args[0], 0};
92
93     int pid = fork();
94     if (pid == 0) { // child
95         exec2(path, argv, stacksize); // TODO : 출력 string 처리
96         printf(1, "(path:%s) execution failed\n\n", path);
97     }
98 }

```

exec2 함수를 통해 path에 위치한 실행 파일을 실행시킵니다.

이 때, fork를 통해 새로운 프로세스가 exec2를 실행하게 하여 pmanager가 멈추지 않도록 합니다.

또한 고민을 했는데,

exec2 된 프로세스 P의 부모는 현재 process manager입니다. fork를 process manager가 진행 했으니 당연한 결과입니다. 그런데, pmanager가 종료되었을 경우 P는 고아가 되어 init process의 child가 됩니다. 그리고 정상 종료 후 init process에게 자원을 회수받습니다.

이때, init process에서는 zombie!를 출력합니다만, process manager가 종료 시 P의 종료를 기다리는 것도 이상하다고 생각했으며, P의 부모를 처음부터 init process로 바꾸는 것 또한 fork를 process manager가 하였으니 의미 상 이상하다 판단하여, P가 고아가 되어 init process의 child가 되는 케이스를 자연스럽다고 생각했습니다. 그래서 해당 케이스를 따로 핸들링하지 않았고 해당 케이스에서는 zombie!가 출력될 것입니다.

- memlim

```
99     else if (!strcmp(cmd, "memlim")) {
100         if (arglen < 2) return -1;
101
102         int pid = atoi(args[0]);
103         int limit = atoi(args[1]);
104
105         if (setmemorylimit(pid, limit) < 0) {
106             printf(1, "(pid:%d) memory limitation failed\n\n", pid);
107         } else {
108             printf(1, "(pid:%d) memory limitation success\n\n", pid);
109         }
110     }
```

setmemorylimit 함수를 통해 pid process의 memory size를 제한합니다.

```

592 // TODO : setmemorylimit
593 int
594 setmemorylimit(int pid, int lim)
595 {
596     struct proc *p;
597
598     acquire(&tsynclock);
599
600     if (lim < 0) goto bad;
601     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
602         if(p->pid == pid)
603             goto limit;
604     goto bad;
605
606 limit:
607     if (p->sz >= lim)
608         goto bad;
609     p->szlim = lim;
610
611     tsync(p);
612     release(&tsynclock);
613     return 0;
614
615 bad:
616     release(&tsynclock);
617     return -1;
618 }

```

process의 sz를 확인 후, szlim을 갱신합니다.

Light-weight Process (LWP)

- thread

```

37 // Per-process state
38 struct proc {
39     uint sz; // Size of process memory (bytes)
40     uint szlim; // Size limit of process memory
41     pde_t* pgdir; // Page table
42     char *kstack; // Bottom of kernel stack for this process
43     enum procstate state; // Process state
44     int pid; // Process ID
45     struct proc *parent; // Parent process
46     struct trapframe *tf; // Trap frame for current syscall
47     struct context *context; // switch() here to run process
48     void *chan; // If non-zero, sleeping on chan
49     int killed; // If non-zero, have been killed
50     struct file *ofile[NFILE]; // Open files
51     struct inode *cwd; // Current directory
52     char name[16]; // Process name (debugging)
53
54     uint spgn; // stack page num
55     void* retval;
56     int tid;
57     struct proc *tmain; // Parent process
58     struct proc *next; // For semaphore
59 };

```

thread의 정상 작동을 위한 여러 변수가 추가되었습니다.


```
2023_ele3021_2020003309 > project02 > C defs.h
```

```
1  struct buf;  
2  struct context;  
3  struct file;  
4  struct inode;  
5  struct pipe;  
6  struct proc;  
7  struct rtcdate;  
8  struct spinlock;  
9  struct sleeplock;  
10 struct stat;  
11 struct superblock;  
12 struct semaphore;  
13 typedef int thread_t;
```

defs.h에 thread_t를 선언합니다. 해당 값은 tid와 똑같이 사용됩니다.

- thread create

```
705 int  
706 tcreate(thread_t *thread, void *(*start_routine)(void *), void *argv)  
707 {  
708     int i;  
709     struct proc *np;  
710     struct proc *curproc = myproc();  
711  
712     uint sz, sp, ustack[3+MAXARG+1];  
713  
714     // alloc proccess, same pid  
715     if((np = allocproc()) == 0)  
716         return -1;  
717     np->tid = np->pid;  
718     np->pid = curproc->pid;  
719  
720     // clone : copy state  
721     np->pgdir = curproc->pgdir;  
722     np->spgdn = 1;  
723     *np->tf = *curproc->tf;  
724     np->parent = curproc->parent;  
725  
726     np->tmain = tmainget(curproc);  
727 }
```

user가 thread_create 함수를 호출하면, tcreate가 호출됩니다.

생성되는 thread는 caller의 pgdir를 그대로 승계받습니다.

```

727
728 // set personal stack
729 acquire(&tsynclock);
730
731 sz = curproc->sz;
732
733 if (curproc->szlim && sz + 2*PGSIZE > curproc->szlim)
734     goto bad;
735
736 sz = PGROUNDUP(sz);
737 if((sz = allocuvn(np->pgdir, sz, sz + 2*PGSIZE)) == 0)
738     goto bad;
739 clearpteu(np->pgdir, (char*)(sz - 2*PGSIZE));
740
741 np->sz = sz;
742 np->szlim = curproc->szlim;
743
744 tsync(np);
745
746 release(&tsynclock);
747

```

thread간 sync를 맞추기 위해 lock 을 건 후, 해당 thread의 sz를 조정합니다.

기본적으로 가드 페이지 하나와 스택 페이지 하나를 할당받습니다.

szlim 은 기본적으로 caller의 szlim을 승계받습니다.

tsync는 thread sync에서 설명드리겠습니다.

- thread exit

```

790 void
791 texit(void *retval)
792 {
793     struct proc *curproc = myproc();
794
795     if (retval) curproc->retval = retval;
796
797     exit_basic(curproc);
798
799     // tmain might be sleeping in wait().
800     wakeup1(curproc->tmain);
801
802     curproc->state = ZOMBIE;
803     sched();
804     panic("zombie exit");
805 }

```

user가 thread_exit를 호출하면, texit가 호출됩니다.

retval을 먼저 저장한 후, exit 로직을 실행합니다.

자원 회수를 위해 main thread를 호출합니다.

```

254 void
255 exit_basic(struct proc *curproc)
256 {
257     int fd;
258     struct proc *p;
259
260     if(curproc == initproc)
261         panic("init exiting");
262
263     // Close all open files.
264     for(fd = 0; fd < NOFILE; fd++){
265         if(curproc->ofile[fd]){
266             fileclose(curproc->ofile[fd]);
267             curproc->ofile[fd] = 0;
268         }
269     }
270
271     begin_op();
272     iput(curproc->cwd);
273     end_op();
274     curproc->cwd = 0;
275
276     acquire(&ptable.lock);
277
278     // Pass abandoned children to init.
279     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
280         if(p->parent == curproc){
281             p->parent = initproc;
282             if(p->state == ZOMBIE)
283                 wakeup1(initproc);
284         }
285     }
286 }
287

```

기본적인 exit 로직입니다.

- thread join

```

807 int
808 twait(thread_t thread, void **retval)
809 {
810     struct proc *p;
811     struct proc *curproc = myproc();
812
813     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
814         if(p->tid == thread){
815             break;
816         }
817     }
818     if (p >= &ptable.proc[NPROC])
819         return -1;
820
821     acquire(&ptable.lock);
822     for(;;){
823         if(p->state == ZOMBIE){
824             *retval = p->retval;
825             freeproc(p, 0);
826             release(&ptable.lock);
827             return 0;
828         }
829
830         // No point waiting
831         if(p->tmain != curproc || curproc->killed){
832             release(&ptable.lock);
833             return -1;
834         }
835
836         // Wait for children to exit.
837         sleep(curproc, &ptable.lock);
838     }
839 }

```

user가 thread_join을 호출하면 twait 함수가 호출됩니다.

thread_t 형식으로 받아온 thread 변수와 tid를 비교하여 타겟 thread를 찾은 후, 해당 thread가 종료되면 자원을 회수합니다.

```

310 void
311 freeproc(struct proc *p, int flag)
312 {
313     kfree(p->kstack);
314     p->kstack = 0;
315
316     if (flag) freevm(p->pgdir);
317
318     p->pid = 0;
319     p->parent = 0;
320     p->name[0] = 0;
321     p->killed = 0;
322
323     p->spgpn = 0;
324     p->retval = 0;
325     p->tid = 0;
326     p->tmain = 0;
327     p->next = 0;
328
329     p->state = UNUSED;
330 }

```

기본적인 자원 회수 로직입니다.

thread의 자원을 회수할 경우 pgdir의 자원은 회수하지 않습니다.

- thread clear

```
842 int
843 tclear(void)
844 {
845     struct proc *p;
846     struct proc *curproc = myproc();
847
848     acquire(&table.lock);
849
850     for(p = table.proc; p < &table.proc[NPROC]; p++)
851         if(p->pid == curproc->pid && p != curproc){
852             p->state = CLEANING;
853             release(&table.lock);
854             exit_basic(p);
855             freeproc(p, 0);
856         }
857
858     release(&table.lock);
859
860     curproc->tid = 0;
861     curproc->tmain = 0;
862
863     return 0;
864 }
865
```

caller와 관계된 thread를 모두 정리합니다.

원래 로직이 약간 달랐으나, Trouble Shooting에서 얘기해드리겠습니다.

타겟 thread의 state를 CLEANING으로 바꿔 스케줄링되지 않도록 합니다.

- thread sync

```
867 void
868 tsync(struct proc *curproc)
869 {
870     struct proc *p;
871     // struct proc *tmain = tmainget(curproc);
872
873     for(p = table.proc; p < &table.proc[NPROC]; p++)
874         if(p->pid == curproc->pid) {
875             p->sz = curproc->sz;
876             p->szlim = curproc->szlim;
877         }
878 }
```

관계된 thread들의 정보를 매개변수 curproc의 정보로 동기화시킵니다.

해당 함수는 sz를 조정하거나 읽는 과정에서 쓰입니다.

Result

- thread test

```
init: starting sh
$ thread_test
Test 1: Basic test
Thread 0 start
Thread 0 end
Thread 1 start
Parent waiting for children...
Thread 1 end
Test 1 passed

Test 2: Fork test
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Child of thread 0 start
Thread 4 start
Child of thread 2 start
Child of thread 1 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Thread 0 end
Child of thread 2 end
Child of thread 3 end
Child of thread 4 end
Thread 2 end
Child of thread 1 end
Thread 3 end
Thread 4 end
Thread 1 end
Test 2 passed

Test 3: Sbrk test
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Test 3 passed

All tests passed!
^C
```

조교님께서 올려주신 thread_test 코드입니다.

- thread exit

```
All tests passed:
$ thread_exit
Thread exit test start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Thread 0 start
Exiting...
^C
```

조교님께서 올려주신 thread_exit 코드입니다.

- thread kill

```
$ thread_kill
Thread kill test start
Killing process 34
This code should be executed 5 times.
This code should be executed 5 times.
This code should be executed 5 times.
This code should be executed 5 times.
This code should be executed 5 times.
Kill test finished
```

조교님께서 올려주신 thread_kill 코드입니다.

- thread exec

```
$ thread_exec
Thread exec test start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 0 start
Thread 4 start
Executing...
Hello, thread!
```

조교님께서 올려주신 thread_exec 코드입니다.

- thread recall

```

$ threadtest
Test 0: recall 1 test
Thread 1-0 start
Thread 1-1 start
Thread 1-2 start
Thread 1-3 start
Thread 1-4 start
Test 1-0: recall 2 test
Test 1-1: recall 2 test
Test 1-2: recall 2 test
Error joining thread 0 in test 1
Test 1-3: recall 2 test
Test 1-4: recall 2 test
Test failed!
Error joining thread 0 in test 1
Thread 2-0 start
Error joining thread 0 in test 1
Test failed!
Thread 2-1 start
Test failed!
Thread 2-2 start
Thread 2-0 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-4 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-4 start
Thread 2-3 start
Thread 2-4 start
Thread 2-0 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-0 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-4 start
Thread 2-4 start

```

제가 만든 recall 테스트 코드입니다. threadtest에 존재합니다.

thread를 재귀적으로 생성한 후, thread가 exit 함수를 통해 비정상 종료를 합니다. Test failed 출력 후, 차례대로 Thread가 종료됩니다. Thread start 문이 Test failed 후 출력되는 부분에 대해서도 Trouble Shooting에서 설명드리겠습니다.

Trouble Shooting

- xv6 재부팅 이슈

개발 과정에서서 main thread 연결 혹은 parent 연결이 잘못되면 xv6가 재부팅되는 현상을 발견하였습니다. 어째서 재부팅되는지는 정확히 알아낼 수 없었으나, main thread 연결과 자원 회수 과정을 치밀하게 연결해주시니 재부팅이 되지 않았습니다.

초기에는 parent에 main thread를 저장하는 방식으로 main thread를 저장했으나 안전하게
게 tmain을 추가하는 방식으로 바꾸었습니다.

- thread sbrk

조교님께서 주신 thread_test의 sbrk test만 가져와서 main loop를 통해 100번 연속 실행 시켜보며 존재하는 크리티컬 섹션을 처리했습니다.

계속 10번 중 한번 꼴로 실패를 해서 파헤쳐보니 umalloc 파일에 freep가 크리티컬 섹션이었습니다. 커널 부분에서 문제가 없고, 유저 단 코드에서 존재하는 문제이기 때문에 고쳐야 하나 싶었는데, 세마포어 구현을 통해 해결했습니다.

```
26 void
27 free(void *ap)
28 {
29     if (!sema.init) init_semaphore(&sema, 1);
30
31     p_semaphore(&sema);
32
33     Header *bp, *p;
34
35     bp = (Header*)ap - 1;
36     for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
37         if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
38             break;
39     if(bp + bp->s.size == p->s.ptr){
40         bp->s.size += p->s.ptr->s.size;
41         bp->s.ptr = p->s.ptr->s.ptr;
42     } else
43         bp->s.ptr = p->s.ptr;
44     if(p + p->s.size == bp){
45         p->s.size += bp->s.size;
46         p->s.ptr = bp->s.ptr;
47     } else
48         p->s.ptr = bp;
49     freep = p;
50
51     v_semaphore(&sema);
52 }
```

```

94 void*
95 malloc(uint nbytes)
96 {
97     if (!sema.init) init_semaphore(&sema, 1);
98
99     Header *p, *prevp;
100     uint nunits;
101
102     p_semaphore(&sema);
103
104     nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
105     if((prevp = freep) == 0){
106         base.s.ptr = freep = prevp = &base;
107         base.s.size = 0;
108     }
109     for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
110         if(p->s.size >= nunits){
111             if(p->s.size == nunits)
112                 prevp->s.ptr = p->s.ptr;
113             else {
114                 p->s.size -= nunits;
115                 p += p->s.size;
116                 p->s.size = nunits;
117             }
118             freep = prevp;
119
120             v_semaphore(&sema);
121             return (void*)(p + 1);
122         }
123         if(p == freep){
124             if((p = morecore(nunits)) == 0){
125                 v_semaphore(&sema);
126                 return 0;
127             }
128         }
129     }
130 }
131

```

따라서 malloc, free 함수 부분이 병렬적으로 실행되지 않아 다소 느려졌지만 크리티컬 섹션을 처리할 수 있었습니다. sbrk test 100번 모두 통과했습니다.

그 이전에 sz 가 존재하는 모든 부분에 tsyncklock을 씌우는 등 크리티컬 섹션 가능성이 있는 부분을 모두 핸들링하면서, 메모리 관리에 관한 이해도를 높일 수 있었습니다... (장장 4일의 여정... user code에 문제가 있을 줄 상상도 못했습니다..)

- tclear deadlock

```

841 // 현재 프로세스를 제외하고, 관련된 스레드를 정리합니다.
842 int
843 tclear(void)
844 {
845     struct proc *p;
846     struct proc *curproc = myproc();
847
848     acquire(&ptable.lock);
849
850     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
851         if(p->pid == curproc->pid && p != curproc){
852             p->state = CLEANING;
853         }
854
855     release(&ptable.lock);
856
857     curproc->tid = 0;
858     curproc->tmain = 0;
859
860     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
861         if(p->state == CLEANING){
862             exit_basic(p);
863             release(&ptable.lock);
864             freeproc(p, 0);
865         }
866
867     return 0;
868 }
869

```

tclear 수정 전 함수입니다.

이렇게 타겟 thread를 모두 스케줄러에 올라가지 않도록 만든 다음 한번에 자원을 회수하니 데드락이 걸렸습니다. 문제는 log.c 함수에 있었습니다.

```

271 begin_op();
272 iput(curproc->cwd);
273 end_op();
274 curproc->cwd = 0;

```

exit_basic에서는 begin_op를 호출합니다.

```

124 // called at the start of each FS system call
125 void
126 begin_op(void)
127 {
128     acquire(&log.lock);
129     while(1){
130         if(log.committing){
131             sleep(&log, &log.lock);
132         } else if(log.lh.n + (log.outstanding+1)*MAXOPBLOCKS > LOGSIZE){
133             // this op might exhaust log space; wait for commit.
134             sleep(&log, &log.lock);
135         } else {
136             log.outstanding += 1;
137             release(&log.lock);
138             break;
139         }
140     }
141 }

```

log.lh.n + (log.outstanding+1)*MAXOPBLOCKS > LOGSIZE 의 값이 true일 경우 caller 는 sleep 상태에 빠지며, 다른 process가 log space를 되돌려주면 다시 일어나 작업을 진행해야 합니다.

하지만 log space 자원을 지닌 다른 process가 스케줄러에 올라가지 못하므로 log space 자원을 되돌려주지 못합니다. 따라서

```

With test finished
$ threadtest
Test 0: recall 1 test
Thread 1-0 start
Thread 1-1 start
Thread 1-2 start
Thread 1-3 start
Thread 1-4 start
Test 1-0: recall 2 test
Test 1-1: recall 2 test
Test 1-2: recall 2 test
Error joining thread 0 in test 1
Test 1-3: recall 2 test
Test 1-4: recall 2 test
Test failed!
Error joining thread 0 in test 1
Thread 2-0 start
Error joining thread 0 in test 1
Test failed!
Thread 2-1 start
Test failed!
Thread 2-2 start
Thread 2-0 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-4 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-4 start
Thread 2-3 start
Thread 2-4 start
Thread 2-0 start
Thread 2-1 start
Thread 2-2 start
Thread 2-3 start
Thread 2-4 start
Thread 2-4 start

```

다음과 같이 자연스럽게 log space를 되돌려 줄 수 있도록

```
842 int
843 tclear(void)
844 {
845     struct proc *p;
846     struct proc *curproc = myproc();
847
848     acquire(&ptable.lock);
849
850     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
851         if(p->pid == curproc->pid && p != curproc){
852             p->state = CLEANING;
853             release(&ptable.lock);
854             exit_basic(p);
855             freeproc(p, 0);
856         }
857
858     release(&ptable.lock);
859
860     curproc->tid = 0;
861     curproc->tmain = 0;
862
863     return 0;
864 }
865
```

프로세스를 하나씩 하나씩 자원을 회수했습니다.